



Estructuras de Datos

Grado en Ingeniería en Informática
Examen de Enero de 2022 (duración 3h)

Nombre:

Grupo:

Pregunta 1 (2p): Contestar V/F las siguientes cuestiones, teniendo en cuenta que una respuesta incorrecta invalida una correcta.

[] De las dos definiciones siguientes, el dato de q se establece en la pila y el de p en el heap:

```
void lacosa () {  
    int q = 3;  
    int *p = new int; *p = 5;  
    ...  
}
```

[] Si no se libera la memoria de p en la función anterior *lacosa()*, entonces se produce un desbordamiento o overflow.

[] Si la clase A tiene una composición a muchos con la clase B y con la clase C , comportándose ambas de forma dinámica, y la clase C tiene una asociación a muchos con la clase B , entonces no es buena idea usar un vector de STL para la relación entre A y B .

[] El constructor copia de la clase *ListaEnlazada*< T > no necesita destructor si se usa en una asociación.

[] Una matriz dispersa y un grafo implementado mediante listas invertidas tienen la misma definición usando STL.

[] Aunque insertar/borrar un dato por el final en un deque< T > y en un vector< T > se considera $O(1)$ es mejor usar un vector< T > para estos casos.

[] El orden de un árbol representa el número de hijos máximo que tiene cada nodo, menos en el árbol B que siempre debe tener ese número de hijos.

[] Ocurre que al intentar insertar los datos v y w en una tabla hash cerrada ocurre que $h(v) == h(w)$, por lo que ambos datos podrían realizar el mismo recorrido para insertarse. Esto se llama agrupamiento primario.

[] Si se utiliza un fichero de datos que mantiene una pila de borrados, la reutilización de una posición de dicho fichero para insertar un nuevo dato se hace con solo tres accesos.

[] Si esta sentencia es válida:

```
int nuevoValor = 7;  
it=mieedd.find(clave);  
(*it).second[i] = &nuevoValor;
```

Entonces *mieedd* puede tener esta definición:

```
map <int, vector<int*> >mieedd;
```

Pregunta 2 (1.5p):

Insertar (I) y borrar (B) la siguiente secuencia de datos en un árbol B de orden 3. Se pueden insertar/borrar más de un dato al mismo tiempo siempre que no afecte a la estructura del árbol.

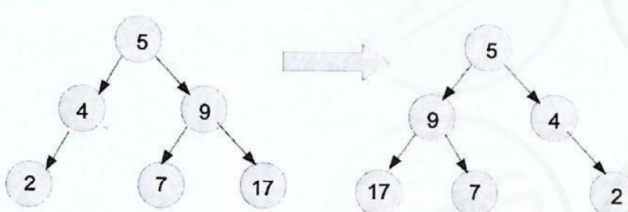
I(3), I(5), I(20), I(30), I(40), I(50), I(15), I(75), I(45), I(47) B(50), B(40)

Pregunta 3 (3p):

(a) Implementar el constructor de la lista simplemente enlazada y circular a partir de un vector dinámico. El vector dinámico está ordenado de menor a mayor y puede tener algún dato repetido. La lista circular debe tener los mismos datos que el vector y de forma ordenada pero no puede tener datos repetidos. El último dato del vector corresponde con el dato apuntado por la cola de la lista. No se deben usar iteradores ni llamada a otras funciones.

```
ListaEnlazadaC<T>::ListaEnlazadaC (const  
VDinamico<T> &v);
```

(b) Implementar la función *ABB::reverse()*, capaz de intercambiar todos los nodos de árbol *ABB* de manera que quede como en la figura:



Pregunta 4 (3.5p):

El gobierno central ha decidido gestionar directamente los datos COVID después del retraso en la comunicación de las cifras por parte de las comunidades autónomas. Ha pensado en un mecanismo de comunicación directo con los centros de salud y hospitales para obtener las cifras en tiempo real. Para ello ha reutilizado una clase *Panel*< T > que almacena datos de forma matricial y que le sirve a otra parte del sistema (que no se implementa aquí) para visionar los datos en una web. Lo que se va a representar en todo momento es la acumulación de positivos, ingresados y de éstos, cuantos están en la UCI. Los datos que se añaden en cada caso son los nuevos casos que quedan almacenados de forma acumulativa con los anteriores. Las altas hospitalarias no se contemplan.

DatoMensual: Es a lo que se instancia cada una de las casillas del panel. En cada una de ellas se acumulan los positivos del mes, los ingresados, y de éstos, cuantos están en la UCI. Añadid getters y setters si son necesarios.



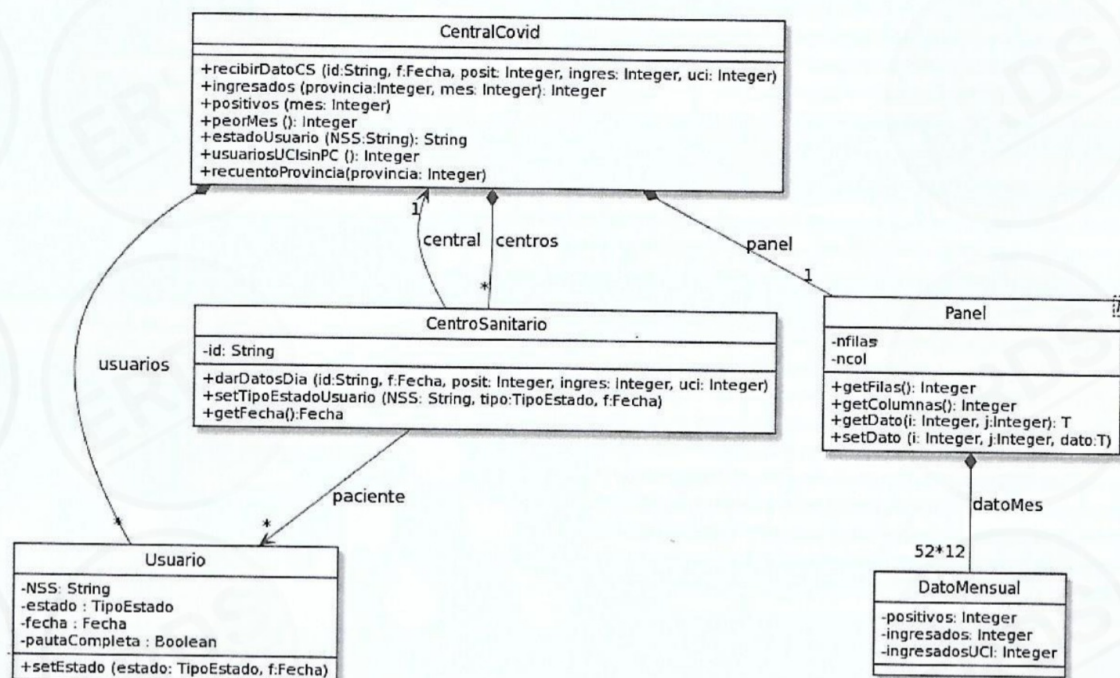
Esta obra pertenece a ERDS bajo la licencia de Creative Commons Reconocimiento-NoComercial-SinObraDerivada 4.0 Internacional

CentroSanitario: Son hospitales, clínicas o centros de salud. Cada noche envían a la central el conteo del día con la función *darDatosDia()*, que hace llegar a la central su conteo diario particular. La central sabe la provincia a la que pertenece porque el identificador de cada centro sanitario tiene codificado en sus dos primeros caracteres el número de la provincia: 00-Álava; 02-Albacete, etc. Conoce en todo momento la hora del sistema mediante la función *getFecha()* que devuelve un objeto de tipo *Fecha*. La clase *Fecha* tiene una función de la que se obtiene el mes con *getMes():Integer*. El centro sanitario también establece el estado del *paciente* si da positivo o queda ingresado con la función *setEstadoUsuario()*.

Panel< T >: Es un panel genérico que sirve para guardar datos y que tiene *nfilas* y *ncol* columnas. El panel sirve para mantener los datos actualizados en tiempo real y para que otra parte del sistema los visualice en una web de acceso público (que no se implementa aquí). Almacenará en cada fila los datos de cada una de las 52 provincias, y en las columnas cada uno de los últimos 12 meses (1=Enero, 2=Febrero, etc). Como es genérica, sólo permite acceder a una posición para leer datos con *getDato()* o escribirlo con *setDato()*.

Usuario: Es cada uno de los usuarios del sistema de salud. Tiene su NSS y un estado indicando si alguna vez ha sido positivo, ingresado o en UCI, es decir, el *TipoEstado*={noDato, Positivo, Ingresado, ingresadoUCI}. Se almacenan siempre como datos históricos, nunca se borran o resetean. Tiene también un atributo que dice si tiene o no la pauta completa. Cuando se actualiza el estado del usuario se pone la fecha de esa actualización.

CentralCovid: Representa al sistema de gestión a nivel nacional. Almacena y gestiona a todos los usuarios y a todos los centros sanitarios. Usa un panel de control con la funcionalidad vista anteriormente donde guarda por filas y columnas los datos directos de las provincias en los últimos 12 meses. Recibe todos los días y de cada centro sanitario la información sobre los nuevos infectados e ingresados con la mediante la función *recibirDatoCS()*. Esta información se usa para actualizar el panel incrementando la casilla correspondiente a la provincia y al mes de la fecha que recibe. La clase también ofrece otra serie de funciones que dan toda la información que se necesita. Por ejemplo, se pueden saber todos los positivos covid de un mes concreto a nivel nacional con *positivos()*, el total de usuarios que hay en las ucis sin pauta completa en los últimos 12 meses con *usuariosUCIsinPC()* o finalmente el mes que más nuevos ingresos hubo a nivel global con *peorMes()*. También puede conocer el estado de cualquier usuario con *estadoUsuario()* o el recuento de positivos de una provincia en los últimos 12 meses con *recuentoProvincia()*.



Esta obra pertenece a ERDS bajo la licencia de Creative Commons Reconocimiento-NoComercial-SinObraDerivada 4.0 Internacional